

Technical Design Document (TDD)

InfusioTech Restaurant Ordering SaaS | Version 1.0 | 2 October 2026

Purpose

Defines the implementation architecture and technical boundaries for the InfusioTech multi-tenant restaurant ordering SaaS.

Architecture Overview

- Customer ordering: responsive Next.js web/PWA; no customer app installation required.
- Restaurant operations: one Flutter app for Android/iOS, with role-aware navigation for owner, manager and staff.
- Platform services: Supabase PostgreSQL, Auth, Storage, Realtime and Edge Functions/server APIs; Vercel or equivalent hosting for web.
- Payments: gateway adapter (initial candidate Razorpay); verify all payment outcomes server-side.
- Platform operations: separate Super Admin area with tightly controlled cross-tenant access.

System Boundaries

- Customer browser must never be trusted for price, tenant authorization or payment state.
- Mobile clients call authenticated APIs; privileged operations run server-side.
- Realtime is a delivery mechanism, not an authorization substitute. Revalidate access when subscribing and when handling sensitive mutations.

Multi-Tenant Design

- Every tenant-owned record includes `tenant_id`; `branch_id` can be added for multi-branch support.
- Resolve tenant membership from authenticated identity; do not trust client-supplied `tenant_id` alone.
- Apply PostgreSQL Row Level Security (RLS) and server-side authorization to every tenant-scoped read/write.
- Use tenant-scoped storage paths and policies; test cross-tenant access explicitly.

Key Flows

- QR/NFC → resolve active restaurant/table → fetch public menu → cart → server validation → create order → payment initiation if selected → verified webhook → status updates.
- Owner invites staff → invitation token validation → staff activates own account → tenant membership and role assigned → permissions checked on each protected action.
- Subscription state → entitlement service → feature gates enforced server-side.

Reliability & Operations

- Use idempotency keys for order/payment initiation and webhook processing.
- Use retries with bounded backoff for transient integrations; dead-letter or flag repeated failures for operator review.
- Maintain backups, restore tests, monitoring, structured logs, error tracking and incident runbooks.
- Prototype may use free tiers; production capacity and uptime depend on selected paid service plans.

Technology Decisions to Confirm

- Flutter; Next.js/React; Supabase/PostgreSQL; Vercel; gateway; push provider; email/SMS/WhatsApp provider; analytics and error-monitoring tools.
- Document exact versions, environments, region, data retention, RPO/RTO and release ownership before production.

Definition of Done

- Architecture diagram reviewed; threat model completed; RLS and API authorization tests pass; secrets are server-side; staging and production are isolated; backup/restore and rollback procedures tested.